

Technical Environment: CalcEngine

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [example-minimal-tech-env-scientific-calculator-api.pdf](#)

Language and Package Manager

- **Python 3.12+**
- **uv** for all package management (no pip, poetry, or conda)
- `pyproject.toml` for all project and tool configuration
- `uv.lock` committed to Git

Web Framework

- **FastAPI** with Pydantic v2 for request/response validation
- **Mangum** to run FastAPI on AWS Lambda

Cloud and Deployment

- **AWS**, single account, `us-east-1`
- **Serverless:** Lambda behind API Gateway (HTTP API type)
- **DynamoDB** for API key storage and usage metering
- **S3 + CloudFront** for documentation site
- **AWS CDK (Python)** for all infrastructure -- no manual console changes

Testing

- **pytest** with pytest-cov (90% line coverage minimum)
- **hypothesis** for property-based math accuracy testing
- **mypy** strict mode for type checking
- **ruff** for linting and formatting

- **moto** for mocking AWS services in tests

Do NOT Use

Prohibited	Reason	Use Instead
<code>eval()</code> , <code>exec()</code> , <code>compile()</code>	Security -- arbitrary code execution	AST-based expression parser
Flask, Django	Project uses FastAPI	FastAPI
requests	Blocks async event loop	httpx
sympy	Too heavy for MVP	Custom expression parser
pandas	Not needed -- single calculations, not dataframes	Standard Python
pip, poetry, pipenv	Project uses uv exclusively	uv
black, flake8, isort	Replaced by ruff	ruff
AWS EC2, ECS, RDS	Serverless model preferred for MVP	Lambda, DynamoDB

Security Basics

- API key auth via `Authorization: Bearer {key}` header
- Keys stored as bcrypt hashes in DynamoDB, never logged in plaintext
- Expression parser uses a character allowlist and AST evaluation -- no dynamic code execution
- Expression length capped at 4,096 characters, nesting depth capped at 100 levels
- TLS 1.2+ enforced, no HTTP endpoints
- Secrets in AWS Secrets Manager, not in environment variables or code

Example Code Pattern

An endpoint should follow this structure:

```

from fastapi import APIRouter, Depends
from pydantic import BaseModel, Field

from calcengine.api.middleware.auth import get_api_key_id
from calcengine.api.models.errors import error_response
from calcengine.api.models.responses import CalculationResponse
from calcengine.engine.errors import CalcEngineError
from calcengine.engine.trigonometry import sin

router = APIRouter()

class SinRequest(BaseModel):
    value: float
    angle_mode: str = Field(default="radians", pattern="^(radians|degrees)$")

@router.post("/v1/trigonometry/sin", response_model=CalculationResponse)
async def calculate_sin(
    request: SinRequest,
    api_key_id: str = Depends(get_api_key_id),
) -> CalculationResponse | dict:
    try:
        result = sin(request.value, angle_mode=request.angle_mode)
        return CalculationResponse(result=result, expression=f"sin({request.value})")
    except CalcEngineError as e:
        return error_response(e)

```

A math function should follow this structure:

```

import math

from calcengine.engine.errors import DomainError

def log_base(value: float, base: float = 10.0) -> float:
    """Compute logarithm of value with given base. Raises DomainError for invalid input."""
    if value <= 0:
        raise DomainError(
            code="DOMAIN_ERROR",
            message=f"Cannot compute logarithm of {value}",
            detail="Logarithm is only defined for positive numbers",
        )
    if base <= 0 or base == 1.0:
        raise DomainError(
            code="DOMAIN_ERROR",
            message=f"Invalid logarithm base: {base}",
            detail="Base must be positive and not equal to 1",
        )
    return math.log(value) / math.log(base)

```

A test should follow this structure:

```
import math
import pytest
from hypothesis import given, strategies as st
from calcengine.engine.errors import DomainError
from calcengine.engine.logarithmic import log_base

def test_log10_of_100() -> None:
    assert log_base(100, 10) == pytest.approx(2.0)

def test_log_of_negative_raises_domain_error() -> None:
    with pytest.raises(DomainError):
        log_base(-5)

@given(st.floats(min_value=1e-300, max_value=1e300, allow_nan=False, allow_infinity=False))
def test_log10_matches_stdlib(x: float) -> None:
    assert log_base(x, 10) == pytest.approx(math.log10(x), rel=1e-14)
```